

Filters in Spring Boot — Quick Revision Notes

1. Why Filters Needed

Common tasks (logging, auth check, timing) shouldn't be repeated in every controller. Filters give one common place to write this logic.

2. Cross-Cutting Concerns

Tasks that apply across the whole app (logging, security, encoding, rate limiting) — not tied to one controller's business logic.

3. Request-Response Flow

`Client → Tomcat → Filters → DispatcherServlet → Interceptors → Controller → Service → Repository` (response goes back in reverse order).

4. What is a Filter

A checkpoint that runs logic **before** and **after** request processing. Uses `chain.doFilter()` to pass control forward.

5. Filters Come from Servlet API

Not a Spring feature — comes from Servlet spec. Spring Boot 3 uses `jakarta.servlet.Filter` (old Boot 2 used `javax.servlet`).

6. Boot 2 vs Boot 3 Imports

Boot 2 → `javax.servlet.*`, Boot 3 → `jakarta.servlet.*`. Always use `jakarta` in new projects.

7. ServletRequest/Response → HTTP versions

Filter gets generic `ServletRequest`/`ServletResponse`; cast to `HttpServletRequest`/`HttpServletResponse` to access HTTP-specific stuff (headers, status, method).

8. DispatcherServlet's Role

It's Spring MVC's main servlet — finds controller, invokes it, converts response. Filters run **before** this, so a Filter doesn't know which controller will handle the request.

9. Creating a Custom Filter

Implement `Filter` interface + annotate class with `@Component` to register it as a bean. Main logic goes inside `doFilter()`.

10. Understanding doFilter()

Takes `request`, `response`, `chain`. Code before `chain.doFilter()` = pre-processing; code after = post-processing.

11. Role of FilterChain

Represents everything left in the pipeline (other filters + `DispatcherServlet`). Filter just calls `chain.doFilter()` without knowing what's next. Not calling it = request stops here.

12. Request/Response Execution Order

Multiple filters execute like nested layers — first filter's "before" code runs first, but its "after" code runs **last** (LIFO order).

13. Reading HTTP Info

Cast request/response to HTTP types, then use methods like `getMethod()`, `getRequestURI()`, `getHeader()`, `setStatus()`, `setHeader()`.

14. Common Use Cases

- **Logging:** log method, URI, IP, user-agent
- **Auth token check:** read `Authorization` header (real auth should use Spring Security)
- **Character encoding:** enforce UTF-8
- **Common headers:** add security/app headers to every response
- **Timing:** measure request duration around `chain.doFilter()`
- **Blocking requests:** reject before reaching controller (e.g. missing API key)
- **Trace/Correlation ID:** generate unique ID per request for tracing logs

15. Blocking Request + Returning JSON

If validation fails, set status + write JSON response directly, then `return` (skip `chain.doFilter()`) — controller never executes.

16. try-finally for Timing/Cleanup

Wrap `chain.doFilter()` in try-finally so cleanup/logging code runs even if an exception occurs downstream.

17. Complete Logging Filter Example

Real filters use SLF4J Logger (not `System.out.println`) to log method, path, IP before request and status+duration after, wrapped in try-finally.

18. Filter Ordering (@Order)

When multiple filters exist, use `@Order(1)`, `@Order(2)` etc. to control sequence. Request goes in that order; response returns in reverse.

19. Where to Put Response Logic

Add must-have headers **before** `chain.doFilter()` (safer) — after downstream processing, response may already be sent/committed.

20. Logging Safety

Never log sensitive data: tokens, passwords, cookies, card info. Log only what's needed, mask sensitive values.

21. When to Use a Filter (Quick Table)

- ✅ Good for: logging, header validation, common headers, blocking early, encoding, timing.
- ❌ Not good for: logic needing controller info (use Interceptor), full security (use Spring Security), business logic (keep in service/controller).

22. Final Mental Model

Filter = reusable HTTP-level checkpoint wrapped around the rest of the request pipeline:

`Request → Filter (before) → rest of chain → Filter (after) → Response`